

UNIT IV MACHINE LEARNING OPERATIONS ARCHITECTURE AND MATURITY MODEL

1. MLOps v2 Architecture

MLOps (Machine Learning Operations) is the set of practices that combines Machine Learning (ML), DevOps, and Data Engineering to automate and streamline the lifecycle of ML models, from development to deployment and monitoring. MLOps v2 focuses on a more end-to-end automation of ML lifecycle.

Key Components of MLOps v2 Architecture:

1. Data Layer

- Collection and storage of structured and unstructured data.
- Tools: Data Lakes, Data Warehouses, Databases.
- Activities: Data preprocessing, cleaning, feature engineering.

2. Model Development Layer

- Building ML models using training datasets.
- Tools: Python, R, TensorFlow, PyTorch, Scikit-Learn.
- Activities: Model experimentation, hyperparameter tuning, validation.

3. Model Deployment Layer

- Deploying trained models into production environments.
- Tools: Docker, Kubernetes, Flask, FastAPI, MLFlow.
- Activities: Model serving, REST APIs, batch or real-time inference.

4. Monitoring & Feedback Layer

- Continuously monitoring model performance in production.
- Activities: Drift detection, error logging, performance metrics.
- Feedback loop feeds into retraining pipeline.

5. Automation & Orchestration

- Ensures smooth end-to-end pipeline management.
- Tools: Kubeflow, Airflow, Jenkins, MLFlow, TFX.
- Automates data ingestion, model training, deployment, and retraining.

MLOps v2 emphasizes continuous integration (CI), continuous deployment (CD), and continuous training (CT) for ML models.

1. Classical Machine Learning Architecture

Classical machine learning (ML) focuses on algorithms that learn patterns from data without explicitly being programmed to perform tasks. It's different from deep learning, which relies on neural networks. Let's break it into components, steps, types, and examples.

A. Components of Classical ML Architecture

1. Data Collection

- Collect raw data from sources like sensors, databases, web scraping, or APIs.
- Example: Images, text, audio, tabular data.
- Tip: Quality of data affects model performance more than the algorithm.

2. Data Preprocessing

- Cleaning: Removing duplicates, missing values, outliers.
- Transformation: Normalization, standardization, encoding categorical features.
- Splitting: Training set, validation set, and test set (common split: 70:15:15).

3. Feature Engineering

- Selecting, creating, or transforming variables (features) to improve model accuracy.

- Techniques:
 - Scaling numerical features.
 - One-hot encoding categorical variables.
 - Dimensionality reduction (PCA, LDA).

4. Model Selection

- Choose the appropriate classical ML algorithm depending on the problem type:
 - **Regression:** Linear Regression, Ridge, Lasso
 - **Classification:** Logistic Regression, SVM, Decision Trees, KNN
 - **Clustering:** K-Means, Hierarchical Clustering, DBSCAN
 - **Dimensionality Reduction:** PCA, t-SNE

5. Training

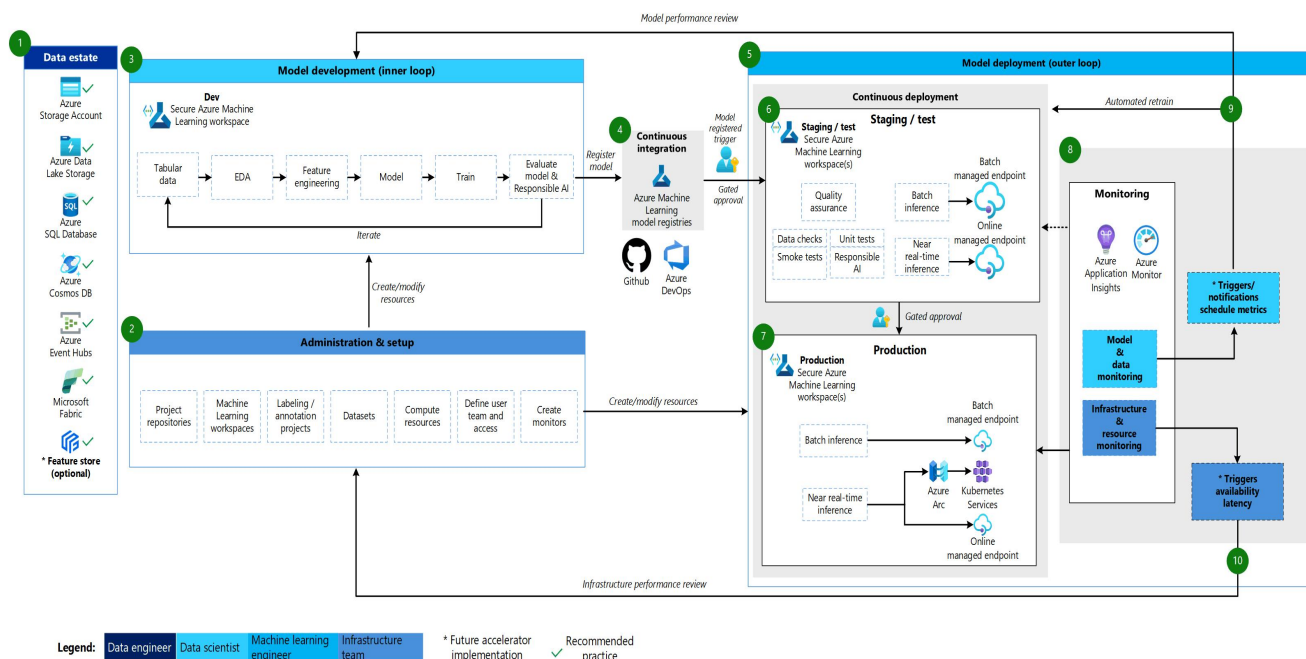
- Fit the model on the training dataset to learn patterns.
- Optimize model parameters using techniques like gradient descent, closed-form solutions.

6. Model Evaluation

- Use metrics to check model performance:
 - Regression: MSE, RMSE, R^2
 - Classification: Accuracy, Precision, Recall, F1-score, ROC-AUC
- Validation techniques:
 - Cross-validation
 - Hyperparameter tuning (Grid Search, Random Search)

7. Deployment

- Integrate the trained model into applications or systems.
- Monitor performance and update the model if performance degrades.



Advantages of Classical Machine Learning

- **Ease of Implementation:** Classical algorithms are simpler to implement and require less computational power compared to deep learning models.
- **Interpretability:** Models like Decision Trees and Logistic Regression provide clear insights into how predictions are made.
- **Cost-Effectiveness:** These models are less resource-intensive, making them suitable for smaller datasets and limited computational environments.

Limitations

- **Scalability:** Classical models struggle with large-scale or high-dimensional data without extensive feature engineering.
- **Adaptability:** They are less effective in handling unstructured data like images or text compared to adaptive or deep learning models.

Use Cases

Classical machine learning is widely applied in areas such as:

- **Fraud Detection:** Logistic Regression and Decision Trees are used to identify fraudulent transactions.
- **Medical Diagnosis:** SVMs and Random Forests help predict diseases based on patient data.
- **Natural Language Processing:** Naive Bayes and Logistic Regression are used for sentiment analysis and spam detection.
- **Data Mining:** Clustering and association rule mining uncover patterns in large datasets.

Practical Implementation Example

Here's a Python example using Scikit-learn to implement a classical machine learning model:

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Sample dataset
X, y = load_iris(return_X_y=True)

# Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialize and train the model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print(f'Model Accuracy: {accuracy * 100:.2f}%')
```

Machine Learning Computer Vision (ML-CV) Architecture

Machine Learning Computer Vision (ML-CV) Architecture refers to the systematic pipeline used to enable machines to interpret, analyze, and understand images or videos using classical machine learning algorithms. Unlike deep learning, ML-CV relies on handcrafted feature extraction and traditional ML classifiers.

1. Image Acquisition

- Images are collected using:
 - Cameras
 - Sensors
 - Public datasets (MNIST, CIFAR, ImageNet)
- Image formats:
 - Grayscale
 - RGB
 - Binary

Purpose: Provide raw visual data for processing.

2. Image Preprocessing

The architecture begins with image acquisition, where images are captured using cameras, sensors, or obtained from standard datasets. These images may be in grayscale, RGB, or other formats depending on the application. Once the images are collected, they undergo image preprocessing, which is a crucial step to improve image quality and consistency. Preprocessing techniques include resizing images to a fixed resolution, removing noise using filters such as Gaussian or median filters, normalizing pixel values, enhancing contrast, and

converting color images into grayscale when necessary. This step ensures that irrelevant variations are reduced and the important visual information is preserved.

Common Preprocessing Techniques

- Resizing images to a standard size
- Noise removal using filters:
 - Gaussian filter
 - Median filter
- Contrast enhancement
- Image normalization
- Grayscale conversion

Why preprocessing is important:

- Reduces noise
- Improves accuracy
- Reduces computational cost

3. Feature Extraction

Classical ML algorithms cannot work directly on raw pixels.

Hence, handcrafted features are extracted.

Technique	Purpose
HOG (Histogram of Oriented Gradients)	Edge & shape detection
SIFT (Scale Invariant Feature Transform)	Keypoint detection
SURF	Faster version of SIFT
LBP (Local Binary Pattern)	Texture analysis
Color Histograms	Color distribution

Technique	Purpose
Gabor Filters	Frequency and texture
The output of this stage is a feature vector	

4. Feature Selection / Dimensionality Reduction

High-dimensional features increase computation and reduce efficiency.

Techniques Used

- PCA (Principal Component Analysis)
- LDA (Linear Discriminant Analysis)
- t-SNE

Advantages:

- Faster training
- Reduced overfitting
- Better generalization

5. Model Training

Extracted features are fed into classical ML algorithms.

Common Algorithms Used

- Support Vector Machine (SVM)
- K-Nearest Neighbors (KNN)
- Decision Tree
- Random Forest
- Logistic Regression

- Naïve Bayes

Training process:

- Model learns patterns from feature vectors
- Parameters are optimized for best performance

6. Model Evaluation

Performance of the trained model is evaluated using test data.

Evaluation Metrics

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix

Validation Techniques

- Train-test split
- K-fold cross validation

7. Prediction / Deployment

- The trained model is used on new unseen images
- Integrated into real-world applications

Applications of ML-CV Architecture

- Face recognition
- Object detection
- Medical image analysis

- Optical Character Recognition (OCR)
- Traffic sign recognition

MACHINE LEARNING NLP ARCHITECTURE

Natural Language Processing (NLP) is a branch of Artificial Intelligence (AI) that helps machines to understand and process human languages either in text or audio form. It is used across a variety of applications from speech recognition to language translation and text summarization.

Machine Learning Natural Language Processing (ML-NLP) Architecture is a structured framework that enables machines to understand, analyze, and generate human language using classical machine learning techniques. This architecture focuses on transforming unstructured text data into numerical representations so that traditional machine learning algorithms can process it. Unlike deep learning-based NLP, ML-NLP relies on manual feature engineering and linguistic knowledge, making it suitable for small to medium-sized datasets and interpretable systems.

The architecture begins with data collection, where textual data is gathered from sources such as documents, emails, social media posts, reviews, or web pages. The collected text is usually unstructured and noisy, containing punctuation, stop words, spelling variations, and irrelevant symbols. Therefore, the next stage is text preprocessing, which plays a vital role in improving model performance. Preprocessing steps include converting text to lowercase, removing punctuation and special characters, eliminating stop words, correcting spelling errors, tokenization (splitting text into words or tokens), and stemming or lemmatization to reduce words to their root forms. These steps help standardize the text and reduce complexity.

After preprocessing, the cleaned text must be converted into a numerical format, as machine learning algorithms cannot directly process raw text. This is achieved through feature extraction or text representation techniques. Common methods include Bag of Words (BoW), Term Frequency–Inverse Document Frequency (TF-IDF), n-grams, and basic word embeddings. These techniques transform text into feature vectors that represent word frequency, importance, or context within the document. The quality of this representation significantly affects the model's accuracy.

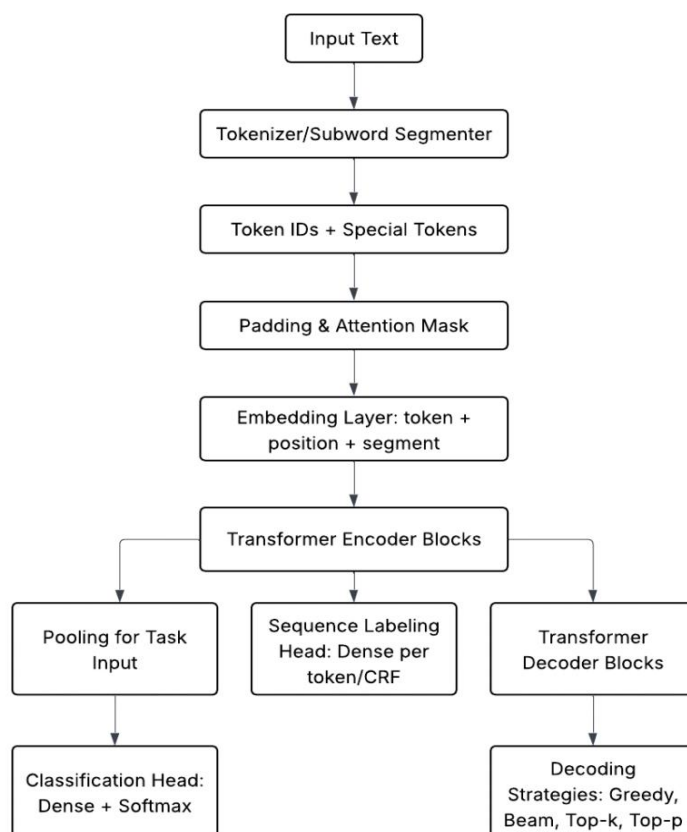
Since text feature vectors can be very high dimensional, feature selection or dimensionality reduction is often applied to improve efficiency. Techniques such as Chi-square test, Information Gain, and Principal Component Analysis (PCA) are used to select the most relevant features while discarding redundant or less important ones. This step reduces computational cost and helps prevent overfitting.

Once the final feature set is prepared, model training is performed using classical machine learning algorithms. Popular algorithms used in ML-NLP include Naïve Bayes, Logistic Regression, Support Vector Machines (SVM), K-Nearest Neighbors (KNN), and Decision Trees. During training, the model learns patterns in the text data, such as word usage and relationships, to perform tasks like classification or prediction.

After training, the model is evaluated using unseen test data to measure its performance. Common evaluation metrics include accuracy, precision, recall, F1-score, and confusion matrix, depending on the NLP task. Proper evaluation ensures that the model generalizes well to new data and performs reliably in real-world applications.

Finally, the trained model is deployed for real-time or batch processing of text data. In deployment, new text inputs pass through the same preprocessing,

feature extraction, and classification pipeline to generate predictions or insights. Machine Learning NLP Architecture is widely used in applications such as spam detection, sentiment analysis, document classification, text categorization, language detection, and basic chatbots.



1. **Input Text:** This is your raw ingredient—the sentence or document you want the machine to understand.
2. **Tokenizer/Subword Segmenter:** Just like chopping vegetables, tokenization cuts text into manageable pieces: words or subwords. This way, the model doesn't have to swallow the whole sentence at once.
3. **Token IDs + Special Tokens:** Each word piece is then turned into a number (an ID) that the computer understands. We also add special

markers like [CLS] or [SEP] to let the model know where things start and stop.

4. **Padding & Attention Mask**

Imagine trying to cook multiple dishes of different sizes in the same oven. Padding makes all text sequences the same length, while the attention mask tells the model which parts are “real” words and which parts are just padding.

5. **Embedding Layer**

Now each token ID is transformed into a rich vector (like turning numbers into flavor profiles). Positional information is also added so the model knows word order.

6. **Transformer Encoder Blocks**

This is the “chef” in the kitchen. It looks at all the tokens together, mixing information using attention. So each word learns not just its own meaning but also how it relates to other words in the sentence.

7. **Task-specific Heads**

Depending on the job, we attach a different “head”:

- **Classification head:** pools everything into one decision (spam vs not spam).
- **Sequence labeling head:** labels each token (NER, POS tagging).
- **Decoder blocks:** used for text generation. They take encoder knowledge plus their own previous words to generate new sentences. Finally, decoding strategies (greedy, beam, top-k) decide how creative or safe the output should be.

Maturity Model and Levels of Maturity

Maturity Model

A maturity model is a structured framework used to assess the capability, effectiveness, and development stage of an organization, process, system, or technology. It helps organizations understand their current state and provides a roadmap for continuous improvement. Maturity models are widely used in areas such as software development, project management, data management, artificial intelligence, cybersecurity, and business process optimization. By following a maturity model, organizations can systematically improve performance, reduce risks, and achieve strategic goals in a planned manner.

Purpose of a Maturity Model

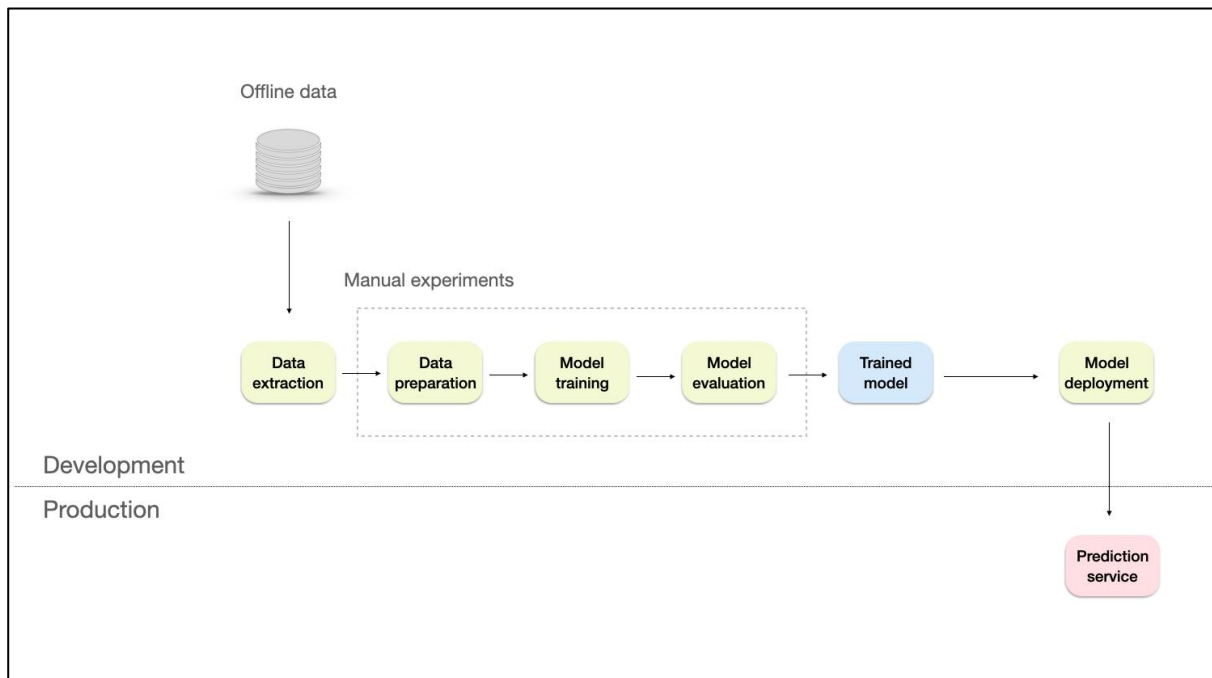
The main purpose of a maturity model is to measure progress and improvement over time. It allows organizations to identify strengths, weaknesses, and gaps in their processes or systems. Maturity models also help in benchmarking against industry standards and best practices. They provide guidance on what actions should be taken to move from a lower maturity level to a higher one. Additionally, maturity models support decision-making, resource allocation, and long-term planning by offering a clear improvement pathway.

Levels of Maturity Model

Most maturity models are divided into **five standard levels**, ranging from initial or ad-hoc practices to optimized and continuously improving systems. Each level represents a distinct stage of capability and control.

- **Level 0: No MLOps**

1. Data scientists work in isolation without regular communication with the larger team.
2. Data collection and preparation are done manually, often using ad-hoc scripts or local datasets.
3. The model release process is entirely manual, which increases the chances of errors and inconsistencies.
4. Implementation depends heavily on data scientist expertise.
5. There is no version control for data, models, or code, making it difficult to reproduce results.



- **Level 1: DevOps no MLOps**

1. Data pipeline automatically gathers data.
2. Release process is manual.
3. Basic integration tests exist for the model.

4. At this level, traditional DevOps practices are applied to application code, but machine learning workflows remain largely manual.
5. There is limited collaboration between ML teams and operations teams.

- **Level 2: Automated training**

1. Data scientists work directly with data engineers to convert experimentation code into repeatable scripts and jobs.
2. Data pipeline automatically gathers data.
3. Compute is managed.
4. Release process is manual.
5. This level improves scalability and consistency of model training.

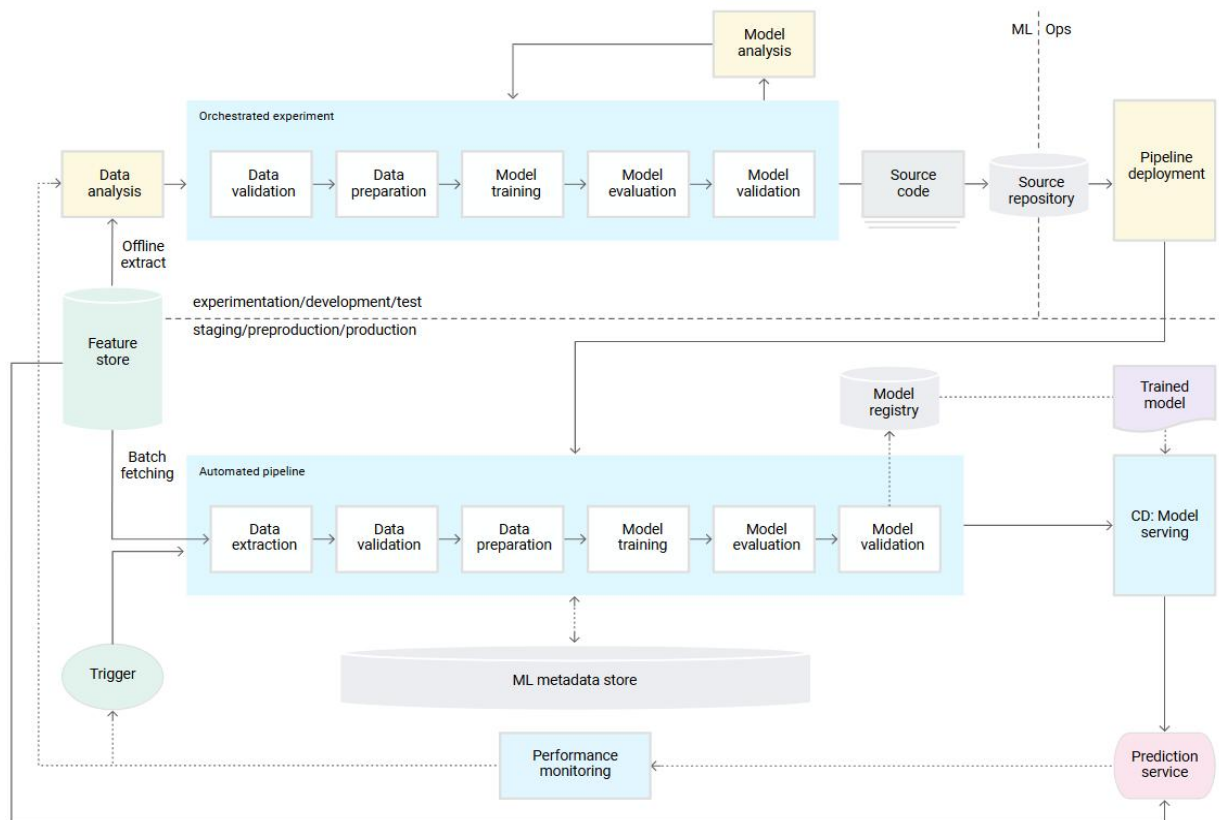
- **Level 3: Automated Model Deployment**

1. Release process is automatic.
2. Each model release includes unit and integration tests
3. Compute is managed.
4. Trained models are automatically packaged and deployed into staging or production environments.

- **Level 4: Full MLOps Automated Retraining**

1. CI/CD pipeline manages releases.
2. Application code has unit and integration tests.
3. Models are automatically retrained when new data arrives or when performance degradation is detected.
4. Continuous monitoring detects data drift, concept drift, and model accuracy issues.

5. Feedback loops enable continuous learning and improvement.



Level	Description	Highlights	Technology
0	No MLOps	• Full machine learning model life cycle is difficult to manage. • Teams are disparate and releases are challenging. • Most systems are nontransparent, with little feedback during and after deployment.	• Builds and deployments are manual. • Model and application testing is manual. • Model performance tracking isn't centralized. • Model training is manual. • Teams use only basic Azure Machine Learning workspace features.
1	DevOps but no MLOps	• Releases are less challenging than Level 0, but rely on data teams for every new model. • Feedback about model performance in production is still limited. • Results are difficult to trace and reproduce.	• Builds are automated. • Application code has automated tests. • Code is version controlled.

2	Automated training	• Training environment is fully managed and traceable. • Model is easy to reproduce. • Releases are manual but easy to implement.	• Model training is automated. • Model training performance tracking is centralized. • Model management is in place. • Machine Learning scheduled or event-driven jobs handle recurring training. • Managed feature store is adopted. • Azure Event Grid life cycle events are emitted for pipeline orchestration. • Environments are managed by using Machine Learning environment definitions.
3	Automated model deployment	• Releases are easy to implement and automatic. • Full traceability exists from deployment back to original data. • Entire environment is managed, including training, testing, and production.	• A/B testing of model performance is integrated for deployment. • All code has automated tests. • Model training performance tracking is centralized. • Artifacts are promoted across workspaces by using Machine Learning registries.
4	Full MLOps automated operations	• Full system is automated and easily monitored. • Production systems provide information about how to improve, and sometimes automatically improve with new models. • System is approaching zero downtime.	• Model training and testing are automated. • Deployed model emits verbose, centralized metrics. • Drift or regression signals trigger automatic retraining by using Event Grid. • Feature materialization health and freshness are monitored. • Model promotion is policy-based and automated by using Machine Learning registries.

Benefits of Using a Maturity Model

Maturity models provide a clear roadmap for growth and improvement. They help organizations reduce risks, improve quality, and optimize resource utilization. By identifying gaps, organizations can prioritize improvement initiatives effectively. Maturity models also support compliance with standards and regulations. Overall, they enable organizations to achieve sustainable performance and competitive advantage.

Applications of Maturity Models

Maturity models are widely applied in various domains such as Software Engineering (CMMI), Project Management (PMMM), Data Analytics, Artificial Intelligence, Cybersecurity, and Business Process Management.